

Probability of Goodness Decision Routing 1.0

A Falsifiable Audit Heuristic, Not a Universal Moral Probability

Shafaet Brady Hussain | Independent researcher, ResearchForumOnline | 18 August 2026

Abstract

I consolidate the Probability of Goodness concept into an auditable agent-governance proposal and evaluate the concrete OpenZero implementation found in the local corpus. The frozen function is not a calibrated probability or a routing gate. It lowercases prompt plus answer, starts at 0.55, adds 0.03 for each of ten positive substrings, subtracts 0.25 for each of seven risky substrings, clamps the result between a configured floor and 0.99, and stores `p_good` in contributed-event metadata. An independent exact mirror and eight synthetic cases reproduce five failure modes: no negation understanding, substring rather than token semantics, answer contamination, a “threshold” that acts as a minimum floor, and positive-word padding of risky text. The release therefore reframes the score as a diagnostic feature that must never independently authorize consequential action. No human preference study, calibration dataset, safety certification, universal ethics claim, or independent validation is reported.

1. Research question

Can Probability of Goodness be expressed as a transparent, falsifiable routing signal without presenting a developer-selected rubric as moral truth? The originating forum paper and later system prompts establish authorial intent: constructive outcomes, visible reasoning, and review for uncertain or harmful actions. They do not establish that goodness is measurable as an objective probability.

The publishable contribution is therefore an engineering boundary. A score may summarize declared features, but policy constraints, tool permissions, reversibility, evidence requirements, and human oversight must retain authority.

2. Frozen implementation evidence

The inspected OpenZero bridge defines `_score_probability_of_goodness(prompt, answer, config)`. It concatenates the prompt and answer, performs case-insensitive substring membership tests, applies fixed increments and decrements, then returns three decimal places. The caller writes the value to a `p_good` metadata field in a knowledge-contribution payload.

No inspected branch uses this value to allow, deny, defer, or escalate an action. The nearby system prompt says to respect a Probability of Goodness threshold, but prompt text is not an enforcement mechanism. More importantly, `P_GOOD_THRESHOLD` is passed to `max(threshold, score)`: increasing it raises low

scores. It is a floor, not a decision threshold.

This distinction is operationally significant. A risk-control name attached to telemetry can create false assurance when the code neither routes nor blocks.

3. Reproducible metamorphic audit

The release supplies an independent exact mirror so the audit does not import a large application with network, configuration, or service side effects. Eight synthetic cases test relationships rather than claiming ground-truth morality.

The audit reproduces all five declared findings:

1. “Explain why we must not phish or steal” scores below a neutral request because negation is ignored.
2. “Anti-phishing” triggers the `phish` substring despite benign context.
3. A refusal answer containing “malware” lowers the combined prompt-answer score, so the answer contaminates the request measurement.
4. Setting the configured threshold to 0.40 forces even a string containing all seven risky signals up to 0.40.
5. Adding positive substrings to risky text raises its score, demonstrating padding sensitivity.

These are not adversarial success-rate statistics. They are deterministic counterexamples to semantic, probabilistic, and enforcement interpretations of the current function.

4. Proposed decision-routing architecture

A safer design separates four layers.

First, hard constraints decide whether an action is prohibited, requires confirmation, or falls outside granted authority. They must not be overridden by a goodness score. Second, evidence features record observable factors such as user authorization, reversibility, data sensitivity, affected parties, uncertainty, and external side effects. Third, a configurable model may estimate a bounded review priority or expected-benefit score. Fourth, a routing policy maps hard constraints, feature uncertainty, and score intervals to `allow`, `defer`, `ask`, `review`, or `deny`.

The audit log should record model/version hash, feature values, missingness, selected route, governing hard rule, override identity, and outcome. A score should be labelled “uncalibrated” until tested on a versioned dataset with independent annotations and reported reliability. Even a calibrated probability would estimate agreement with a defined outcome label, not metaphysical goodness.

5. Evaluation protocol

Future evaluation should pre-register the task distribution, consequence classes, label rubric, annotator instructions, disagreement handling, protected-group analysis where lawful and appropriate, and release thresholds. Split data by scenario family to reduce near-duplicate leakage. Report confusion matrices for each route, false-negative cost, false-positive delay, abstention coverage, calibration error, distribution shift, and override outcomes.

High-impact decisions require domain-specific review and appeal. NIST AI RMF 1.0 frames trustworthy AI characteristics and risk management through GOVERN, MAP, MEASURE, and MANAGE functions [1]. Its Generative AI Profile adds practices for generative-AI risks [2]. These sources support structured risk

governance; they do not validate this heuristic.

6. Threats and misuse cases

Keyword padding can raise a score. Negation, quotation, educational discussion, multilingual input, obfuscation, spelling variation, and context collapse can reverse apparent meaning. Combining prompt and answer confounds intent with model behavior. A high configured floor can erase risk separation. An attacker may target known features, while an operator may over-trust a numeric display.

There is also a governance hazard: values chosen by a developer can encode preferences or uneven burdens while appearing neutral because the output is numeric. The mitigation is not a more impressive equation. It is explicit definitions, contestability, measured error, bounded authority, and auditable fallback behavior.

7. Falsifiable claims and release gates

1. The supplied audit must reproduce all five metamorphic findings across eight cases.
2. The frozen implementation must not be called a routing gate unless code demonstrably consumes it to select a route.
3. `P_GOOD_THRESHOLD` must not be called an acceptance threshold while it remains a lower clamp.
4. Any future “probability” claim requires a defined target event, labelled evaluation set, calibration analysis, and uncertainty report.
5. Any consequential deployment requires hard-rule precedence, least-authority tools, logged overrides, and a tested human-review route.
6. A change in model, rubric, features, or threshold invalidates prior evaluation until rerun.

8. Limitations

The audit mirrors one frozen local implementation and does not prove which revision is deployed publicly. The eight cases are synthetic counterexamples, not a representative benchmark. I did not collect human moral judgments, measure demographic impacts, test multilingual coverage, run user research, or independently validate the system. The proposed architecture is a specification, not a completed safety system. NIST references provide general risk-management context, not endorsement or certification.

9. Conclusion

Probability of Goodness is defensible only as a named, configurable audit heuristic whose target, errors, and authority are explicit. The current implementation is telemetry derived from substrings, not a calibrated probability and not an enforcement gate. The next credible step is to replace suggestive naming with measurable routing outcomes, place hard constraints above scores, and evaluate abstention and error costs on a versioned review corpus.

Evidence ledger

- Frozen implementation: `DEPLOY_OPENZERO/hivemind/bridge.py`, SHA-256 `8b4b16854b9c7052fca14493f2d3d7c4c874f9875bb11d3407d119c40612880f`, function `_score_probability_of_goodness`, plus `p_good` payload use.

- Concept specification: ZERO_CYMATIC_MATH_MEANING.md, SHA-256 0a9e132008550a7fbea46938511423aeb9174fbf8eff2ac1d2efde8ff4d11a4a, Probability of Goodness Gate section.
- Reproducibility package: artifacts/probability-goodness-routing-1.0/.
- Forum predecessor: “Probability of Goodness: A Comprehensive Analysis of Choosing Good Over Bad,” treated as an originating concept note, not external validation.

AI-use disclosure

AI tools assisted corpus search, code inspection, counterexample design, drafting, editing, and packaging. I take responsibility for the claims and release decisions. AI assistance is not peer review or independent validation.

Licensing

The paper is CC BY 4.0. The new audit code is MIT licensed. Referenced application files retain their original terms and are not redistributed.

References

[1] NIST. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, 2023. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>

[2] NIST. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>